



FocusBoost Development Progress Documentation

Date: July 26, 2025

Project: FocusBoost - AI-Powered Children's Attention & Focus Enhancement System

Phase: Foundation Complete - Ready for Session Management



Project Overview

FocusBoost is a web-based system that uses computer vision and machine learning to enhance children's attention and cognitive skills. The system monitors children during study sessions, detects attention levels, and provides adaptive interventions through AI-generated quizzes.

Target Users: Children aged 6-12 and their parents

Core Technology: React + TypeScript frontend, FastAPI + Python backend, Firebase Firestore database



COMPLETED FEATURES



1. Authentication System (100% Complete)

Backend (FastAPI):

- **User Registration** (POST /register)
 - Email validation and password requirements
 - Firebase Auth integration
 - Pydantic schema validation
 - Error handling for duplicate emails
- **User Login** (POST /login)
 - Firebase Auth verification
 - JWT token generation
 - Remember me functionality
 - Custom token creation
- **Password Reset** (POST /forgot-password)
 - Firebase password reset email
 - Error handling and validation
- **User Management** (GET /users)
 - Admin endpoint to list all users

- Complete user metadata retrieval

Frontend (React + TypeScript):

- ☒ **Login.tsx** - Clean UI with validation and error handling
- ☒ **Signup.tsx** - Registration form with password confirmation
- ☒ **ResetPassword.tsx** - Password reset workflow
- ☒ **UserContext.tsx** - Authentication state management
- ☒ **App.tsx** - Route protection and auth flow

Integration:

- ☒ Frontend connects to FastAPI backend (not Firebase directly)
 - ☒ Token storage in localStorage
 - ☒ Automatic token management
 - ☒ Route protection based on authentication state
-

2. Child Profile Management (100% Complete)

Backend (FastAPI):

- ☒ **Create Child** (POST /api/children)
 - Name, age, parentId validation
 - Auto-generated avatars using DiceBear API
 - Random seed generation for unique avatars
- ☒ **Get Children by Parent** (GET /api/children/{parent_id})
 - Filtered by parent ID
 - Complete child data retrieval
 - Proper error handling
- ☒ **Get Child by ID** (GET /api/children/child/{child_id})
 - Individual child profile lookup
 - 404 handling for non-existent children
- ☒ **Update Child** (PUT /api/children/{child_id})
 - Name, age, and avatar seed updates
 - Partial update support
 - Avatar regeneration on seed change

- ☒ **Delete Child** (DELETE /api/children/{child_id})
 - Complete profile removal
 - Cascade considerations handled

Frontend (React + TypeScript):

- ☒ **ProfileManager.tsx** - Child profile CRUD interface
 - Grid view of all child profiles
 - Add new child modal with avatar preview
 - Edit and delete buttons with confirmations
 - Loading states and error handling
- ☒ **EditProfile.tsx** - Child profile editing
 - Form validation and submission
 - Avatar regeneration functionality
 - Success/error feedback
- ☒ **UserContext.tsx** - Child state management
 - API integration for all child operations
 - Optimistic updates for better UX
 - Error handling and loading states

Database Schema:

- ☒ **Children Collection** (Firebase Firestore)

typescript

```
interface Child {  
  id: string;      // Firestore document ID  
  name: string;    // Child's name  
  age: number;     // Age (6-12)  
  parentId: string; // Reference to parent user  
  avatar: string;   // DiceBear avatar URL  
  seed: string;     // Avatar generation seed  
  createdAt: DateTime; // Creation timestamp  
  updatedAt: DateTime; // Last update timestamp  
}
```

3. API Service Layer (100% Complete)

apiService.ts:

- ☒ **HTTP Client Setup** - Centralized fetch wrapper
- ☒ **Authentication Management** - Token storage and headers
- ☒ **Error Handling** - Comprehensive error responses
- ☒ **TypeScript Interfaces** - Matching backend schemas
- ☒ **React Hooks** - useApiCall and useApiMutation for easy usage

Endpoints Implemented:

typescript

// Authentication

POST /register ☒ User registration
POST /login ☒ User login
POST /forgot-password ☒ Password reset

// Child Management

POST /api/children ☒ Create child profile
GET /api/children/{parentId} ☒ Get children by parent
GET /api/children/child/{id} ☒ Get specific child
PUT /api/children/{id} ☒ Update child profile
DELETE /api/children/{id} ☒ Delete child profile

// System

GET /health ☒ Health check
GET / ☒ Root endpoint

4. User Interface (100% Complete)

Design System:

- ☒ **Tailwind CSS** - Consistent styling framework
- ☒ **Component Architecture** - Reusable UI components
- ☒ **Color Scheme** - Child-friendly blues and purples
- ☒ **Responsive Design** - Mobile and desktop compatible

Navigation & Routing:

- ☒ **React Router** - Client-side routing
- ☒ **Protected Routes** - Authentication-based access control

-  **Route Guards** - Automatic redirects for auth states
-  **Loading States** - Smooth transitions during auth checks

User Experience:

-  **Form Validation** - Real-time feedback
 -  **Error Messages** - Clear, actionable error states
 -  **Success Feedback** - Confirmation of successful actions
 -  **Loading Indicators** - Visual feedback during API calls
-

5. Database & Infrastructure (100% Complete)

Firebase Setup:

-  **Firebase Auth** - User authentication service
-  **Firestore Database** - NoSQL document database
-  **Security Rules** - Basic access control (needs enhancement)
-  **Collections Structure** - Users and children collections

Backend Architecture:

-  **FastAPI Framework** - Modern Python web framework
-  **Pydantic Schemas** - Data validation and serialization
-  **CORS Configuration** - Frontend-backend communication
-  **Error Handling** - Comprehensive exception management

Development Environment:

-  **Local Development** - Backend on localhost:8000
 -  **Environment Variables** - Configuration management
 -  **Development Workflow** - fastapi dev main.py setup
-

TESTING STATUS

Verified Working Features:

Authentication Flow:

-  User can register new account

- ☒ Registration redirects to login page
- ☒ User can login with valid credentials
- ☒ Login redirects to /profiles page
- ☒ Invalid credentials show error messages
- ☒ Password reset sends email
- ☒ Logout clears authentication state

Child Management Flow:

- ☒ Authenticated users can view child profiles
- ☒ Users can create new child profiles
- ☒ Avatar generation works correctly
- ☒ Users can edit existing child profiles
- ☒ Avatar regeneration works in edit mode
- ☒ Users can delete child profiles
- ☒ Profile changes persist after page refresh

API Integration:

- ☒ All endpoints return proper responses
- ☒ Error handling works for failed requests
- ☒ Loading states display during API calls
- ☒ Token authentication works correctly
- ☒ CORS allows frontend-backend communication

CURRENT PROJECT STRUCTURE

Backend Structure:

```

app/
├── __init__.py
├── main.py           # FastAPI application
├── settings.py       # Configuration settings
├── account_schema/
│   ├── __init__.py
│   └── schema.py     # Pydantic models
├── user_account/
│   ├── __init__.py
│   └── auth.py       # Authentication logic
├── firebase-credentials.json # Firebase service account
└── requirements.txt   # Python dependencies

```

Frontend Structure:

```

src/
├── components/
│   ├── parent/
│   │   └── ProfileManager.tsx # Child profile management
│   └── shared/
│       └── Logo.tsx          # App logo component
├── context/
│   └── UserContext.tsx       # Global state management
├── pages/
│   ├── auth/
│   │   ├── Login.tsx        # Login page
│   │   ├── Signup.tsx       # Registration page
│   │   └── ResetPassword.tsx # Password reset page
│   ├── EditProfile.tsx      # Child profile editing
│   ├── ParentDashboard.tsx  # Main dashboard
│   └── ProfileSelection.tsx  # Profile selection
├── services/
│   └── apiService.ts         # API client layer
└── App.tsx                  # Main app component with routing

```

TECHNICAL STACK

Frontend:

- **Framework:** React 18 with TypeScript
- **Styling:** Tailwind CSS

- **Routing:** React Router v6
- **State Management:** React Context + useState
- **HTTP Client:** Fetch API with custom wrapper
- **Avatar Generation:** DiceBear API integration
- **Icons:** Lucide React
- **Animations:** Framer Motion

Backend:

- **Framework:** FastAPI (Python)
- **Database:** Firebase Firestore
- **Authentication:** Firebase Auth
- **Validation:** Pydantic
- **CORS:** FastAPI CORS middleware
- **Development Server:** uvicorn

Database Schema:

- **Users:** Managed by Firebase Auth
- **Children:** Custom Firestore collection with parent references



DEPLOYMENT STATUS

Current State:

- ☒ **Development Environment** - Fully functional locally
- ☐ **Production Deployment** - Not yet deployed
- ☐ **CI/CD Pipeline** - Not configured
- ☐ **Environment Management** - Basic .env setup only

Deployment Readiness:

- ☒ Backend can run with fastapi dev main.py
 - ☒ Frontend builds successfully
 - ☒ Environment variables properly configured
 - ☐ Production configuration needed
 - ☐ Database security rules need review
-

KNOWN LIMITATIONS & TODOs

Security:

-  **Firestore Security Rules** - Currently permissive, need tightening
-  **Input Validation** - Basic validation present, could be enhanced
-  **Rate Limiting** - Not implemented
-  **HTTPS** - Not configured for production

Error Handling:

-  **Network Error Recovery** - Basic retry logic missing
-  **Offline Support** - Not implemented
-  **Error Logging** - No centralized error tracking

User Experience:

-  **Loading Performance** - No optimization for large child lists
-  **Accessibility** - Basic ARIA labels missing
-  **Mobile UX** - Responsive but not optimized for mobile

Data Management:

-  **Data Backup** - No backup strategy implemented
 -  **Data Migration** - No versioning strategy
 -  **Data Validation** - Client-side only
-

NEXT PHASE: STUDY SESSION MANAGEMENT

Immediate Next Steps:

1. **Design Session Database Schema**
2. **Implement Session Management APIs**
3. **Build Session UI Components**
4. **Integrate Session Timer Functionality**
5. **Add Basic Session Analytics**

Success Criteria for Next Phase:

-  Children can start and stop study sessions
 -  Session time tracking works accurately
 -  Session history is stored and retrievable
 -  Parents can view session summaries
 -  Session data is properly associated with child profiles
-

PROJECT METRICS

Development Progress:

- **Overall Completion:** ~25% (Foundation complete)
- **Authentication System:** 100% 
- **Child Management:** 100% 
- **Study Sessions:** 0% 
- **ML Integration:** 0% 
- **Quiz System:** 0% 
- **Parent Analytics:** 0% 

Code Quality:

- **TypeScript Coverage:** 100%
- **Error Handling:** Basic implementation
- **Testing:** Manual testing only
- **Documentation:** In progress

Performance:

- **API Response Time:** < 500ms (local)
 - **Page Load Time:** < 2s (local)
 - **Database Queries:** Optimized for current scale
 - **Bundle Size:** Not optimized
-

TEAM & RESOURCES

Current Team:

- **Developer:** Shadhujan Jeyachandran

- **AI Assistant:** Claude (Anthropic)

External Services:

- **Authentication:** Firebase Auth
 - **Database:** Firebase Firestore
 - **Avatar Generation:** DiceBear API
 - **Development Tools:** VS Code, Git
-

DEVELOPMENT NOTES

Key Decisions Made:

1. **Switched from Firebase Auth frontend to FastAPI backend auth** - Better control and consistency
2. **Used DiceBear API for avatars** - Consistent with backend implementation
3. **Implemented comprehensive error handling** - Better user experience
4. **Used TypeScript throughout** - Better code quality and developer experience

Lessons Learned:

1. **API schema consistency is crucial** - Frontend/backend mismatches caused delays
2. **Error handling should be implemented early** - Retrofitting is time-consuming
3. **User context design impacts all components** - Plan state management carefully
4. **Testing authentication flows manually is tedious** - Automated testing needed

Best Practices Established:

1. **Consistent naming conventions** - camelCase frontend, snake_case backend
 2. **Comprehensive error messages** - User-friendly feedback
 3. **Loading states for all async operations** - Better UX
 4. **TypeScript interfaces matching backend schemas** - Type safety
-

ACHIEVEMENTS & MILESTONES

Major Milestones Completed:

-  **M1:** Project setup and initial configuration
-  **M2:** Authentication system implementation
-  **M3:** Child profile management system

-  **M4:** Frontend-backend integration
-  **M5:** Basic user interface completion

Technical Achievements:

-  **Clean Architecture:** Well-separated concerns between frontend/backend
 -  **Type Safety:** Full TypeScript implementation
 -  **Error Handling:** Comprehensive error management
 -  **State Management:** Effective React Context implementation
 -  **API Design:** RESTful, consistent endpoint structure
-

 **Next Documentation Update:** After Study Session Management implementation

 **Last Updated:** July 26, 2025

 **Document Version:** 1.0.0